Technical report, ClearConsent

Plain-language, multilingual and quote-verified research consent forms with Apertus.

- **Track** Track 2B, Own Project
- **Event** Hack Apertus Online, October 2026
- **Team** ClearConsent, Seika Karamatsu
- **Demo** video linked in the submission form, also in the repository as `docs/ClearConsent_demo.mp4`
- **Code** https://github.com/sese0318/clearconsent

1. Summary

People sign research consent forms that they do not fully understand. The forms are long, written in legal and medical language, and usually offered in one language. Readers who use the form's language as a second language, older adults and people with low health literacy are the most likely to miss what matters to them, such as how much time a study takes, what the risks are, what happens to their data and that they can stop at any time.

ClearConsent lets a participant paste a consent form and read it in their own language. Apertus extracts nine topics that ethics guidance asks participants to understand, and copies, for every topic, the sentence of the form that supports its answer. A deterministic verifier then checks that each quoted sentence really exists in the form and that every number in the answer appears in the form. Only verified facts reach the plain-language explanation. Topics the form does not state are not filled in by the model. They become concrete questions to ask the research team. A short teach-back quiz, generated from verified facts only, checks understanding.

Headline result. On six synthetic consent forms in four languages, Apertus 8B answered all 48 stated topics, every one of its 48 quotes was found in the form, and all 20 key facts (amounts, durations, retention periods, contacts) were correct. For the 6 topics that were deliberately missing, the raw model still produced an answer twice (33%). The verifier held back both, so no invented fact reached the reader (0%). One document costs less than a tenth of a US cent with the 8B model (three calls, about 3,300 tokens).

2. Architecture

The full diagram is in `docs/architecture.md`. The project is one Docker container with no third-party Python packages.

Component	What it does	Where it runs
Web UI (`src/clearconsent/static/index.html`)	Paste or load a form, choose a language, read the explanation, facts with quotes and quiz	Browser, served by the container, no external fonts or scripts
Server (`server.py`)	Serves the UI and `POST /api/analyze`	Container, Python 3.12 standard library
Pipeline (`pipeline.py`)	Extract, verify, explain, quiz	Container
LLM client (`llm.py`)	OpenAI-compatible chat calls configured by `LLM_NAME`, `LLM_BASE_URL`, `LLM_API_KEY`	Container
Apertus v1.5 8B or 70B	Language understanding and generation	Any OpenAI-compatible endpoint

Pipeline for one document (three Apertus calls, explanation and quiz run in parallel)

1. **Extract.** Apertus reads the form at temperature 0 and returns JSON with nine keys (purpose, procedures, time commitment, compensation, risks, benefits, data and privacy, voluntary participation and withdrawal,

contact). Each key holds a one-sentence English answer and one sentence copied verbatim from the form in its original language. The prompt asks for an empty answer whenever the form does not state the topic.

2. **Verify (no LLM).** Text is normalized (Unicode NFKC, quotes, whitespace, markdown symbols). A quote passes when it appears in the form, or when the closest window of the form reaches a similarity of at least 0.90. Every number in the answer must also occur in the form. Facts that fail are labeled "needs checking" and are never used as facts downstream.
3. **Explain.** Apertus receives only the verified facts and the list of missing topics, and writes a plain-language explanation at a reading level for a 12-year-old in the chosen language, with fixed sections, ending with "Questions to ask before you sign".
4. **Quiz.** Apertus writes three multiple-choice questions from verified facts. The server keeps a question only if its answer index is valid and it refers to a verified topic.

Target architecture

ClearConsent is deployable **on-premise (a)** and **air-gapped (b)**, and can use a **sovereign Swiss cloud endpoint (c)** without code changes.

- **On-premise and air-gapped.** `docker-compose.airgap.yml` adds a vLLM service that serves local Apertus v1.5 8B weights on an internal Docker network marked `internal: true`, so the model cannot reach the internet. The UI loads no external assets. A hospital or university can run the stack on one GPU server inside its own network, under its own administration.
- **Swiss cloud.** Pointing `LLM_BASE_URL` at an OpenAI-compatible Apertus endpoint hosted in Switzerland is a configuration change only.
- **Demo setup.** The demo and the evaluation used the Public AI gateway (`https://api.publicai.co/v1`), a hosted OpenAI-compatible endpoint for Apertus.

External dependencies. Build time needs the `python:3.12-slim` base image (and the `vllm/vllm-openai` image plus Apertus weights for the air-gapped variant). Runtime needs only the configured LLM endpoint. The app stores no data, keeps no database and does not log document text, which matters because consent forms and participant questions can reveal health conditions.

3. Use of Apertus

- **Model** `swiss-ai/apertus-v1.5-8b` (all results in this report). The code also runs with `swiss-ai/apertus-v1.5-70b`, but the 70B evaluation was rate-limited by the hosted gateway
- **How it is used** inference only (structured extraction, constrained plain-language generation, quiz generation)
- **Where it runs** hosted endpoint for the demo (Public AI gateway), local vLLM for on-premise and air-gapped use

Settings. Extraction uses temperature 0 and at most 1,600 output tokens. Explanation and quiz use temperature 0.2 and at most 900 and 700 tokens. Prompts are in `src/clearconsent/pipeline.py`. No fine-tuning, adapters or quantization were used.

Why Apertus. Consent forms are health-research documents. Research sites want a model they can run themselves, with open weights and documented training data that an ethics committee can inspect. Apertus is fully open and was trained on text in more than 1,800 languages, which covers the four national languages of Switzerland and many languages spoken by people who take part in Swiss research. A small open model that can be self-hosted is what makes this use case deployable at all.

No other model is used at runtime or in the evaluation. Development of the code was assisted by an AI coding assistant (Claude Code). It did not generate any data used for scoring and is not part of the product.

4. Data

`data/forms/` holds six synthetic consent forms written for this project, about fictional studies at fictional institutions, three in English and one each in German, French and Italian. They contain no personal data. Five of the six forms deliberately leave out one or two topics (for example payment, risks, contact or data handling) to test whether the model invents them. `data/gold.json` lists, for each form, the topics that are absent and the key facts (numbers and names) that a correct answer must contain. Forms and labels are released under CC-BY-4.0.

5. Evaluation

Task. For each form, extract the nine topics and compare them with the gold labels. The baseline is the raw Apertus extraction. "Ours" is the same extraction after the verification layer, which is what the participant sees.

Setup	Stated topics answered	Key facts correct	Quotes found in the form	Missing topics answered	Missing topics shown to the reader as fact
Baseline, raw Apertus 8B extraction	48 of 48 (100%)	20 of 20 (100%)	48 of 48 (100%)	2 of 6 (33%)	2 of 6 (33%)
Ours, Apertus 8B plus verification	48 of 48 (100%)	20 of 20 (100%)	48 of 48 (100%)	2 of 6 (33%)	**0 of 6 (0%)**

Extraction returned valid JSON for 6 of 6 forms. Mean extraction size was 1,265 tokens per form, which is about $0.00018 per form at the listed 8B price. Full per-form results are in `docs/eval/results.json` and `docs/eval/results.md`.

Metric definitions. Recall is the share of the 48 stated topics that were answered (raw) or answered and verified (ours). Key-fact recall is the share of gold key facts that appear in verified answers. Quotes grounded is the share of non-empty quotes found in the form. Absent topics answered is the share of the 6 deliberately missing topics for which the model still produced an answer, and absent topics shown as fact is the share that would pass verification and be shown to the participant as a fact. Cost uses the Public AI list prices (8B at $0.10 input and $0.20 output per million tokens, 70B at $0.82 and $2.92).

What the verifier caught. Both unsupported answers sounded plausible and came without a supporting quote. For the MRI form, which never mentions benefits, the model wrote that there are no direct benefits and that the study is for research purposes only. For the Italian form, which never says that participation is voluntary, the model wrote "Participation is voluntary". A participant could easily rely on either sentence. Both were labeled "needs checking" and turned into questions for the research team.

Findings from development.

- The first version of the extraction prompt made Apertus 8B flatten the nested JSON on the German form, so all nine topics were lost (they safely fell back to "not stated"). Adding one example of the expected shape fixed this (6 of 6 forms valid). A lenient parser that recovers each topic from malformed output stays in place as a safety net, and the app reports which parser was used.
- In one development run, the quiz marked "a voucher worth CHF 100" as the correct answer although the form says CHF 50. The server now drops any quiz item whose correct option contains a number that is not in the verified quotes.
- English explanations of the German form and Japanese explanations of the French form were spot-checked and were accurate and readable. The missing topic was correctly turned into a question in both.

Latency and the 70B model. In the app, one document takes about 40 seconds end to end with 8B (three calls, explanation and quiz in parallel, measured on three runs). Repeated identical requests are answered faster by the gateway, so latency from evaluation re-runs is not reported. The same evaluation was started for Apertus 70B, but every request was rejected by the hosted gateway with HTTP 429 (rate limit) during the hackathon window, so this report has no 70B numbers. `make eval` can run it again.

6. Limitations

- The evaluation set is small (6 forms, 54 topic slots) and synthetic. It shows that the pipeline works across four languages but is not a validation on real consent forms.
- The verifier checks that the quoted sentence exists and that numbers match. It does not prove that the answer is a correct reading of the quote, so a wrong interpretation of a real sentence can still pass.
- The explanation step is restricted to verified facts by the prompt, not by a formal check. Generated text in low-resource languages (for example Romansh or Tigrinya) has not been reviewed by native speakers.
- ClearConsent supports the consent conversation and does not replace it. It is not legal or medical advice, and the research team remains responsible for informed consent.
- The air-gapped vLLM configuration was written but not run, because no GPU was available during the hackathon.
- Forms longer than 20,000 characters are truncated.

7. Reproducibility

- `make run` builds the container and starts the app on `http://localhost:8000`. Set `LLM_API_KEY` (and optionally `LLM_NAME`, `LLM_BASE_URL`) in the shell or in `.env` (see `.env.example`).
- `make eval` re-runs the evaluation (8B by default, add 70B with `MODELS="swiss-ai/apertus-v1.5-8b swiss-ai/apertus-v1.5-70b"`) and writes `docs/eval/results.md` and `results.json`.
- Without Docker, `python serve.py` runs the same app with Python 3.10 or newer and no dependencies.
- Every push runs `.github/workflows/smoke.yml` on GitHub Actions. It runs the unit tests (verifier and JSON parsing, no model calls), starts the container with `make run`, and checks the health endpoint, the sample list and the UI.
- Hardware used. A Windows laptop without a GPU. All inference ran on the hosted endpoint. Extraction uses temperature 0, so reruns should be close, but hosted inference is not guaranteed to be bit-for-bit deterministic.
- Exact commit. See the GitHub repository history. The evaluation in this report was run on the code of commit `d85b75e`.

8. Next steps

- Pilot with a research site and an ethics committee on real, published consent templates, with participants who read the form in a second language.
- Add a semantic check in which Apertus judges, quote by quote, whether the answer follows from the quoted sentence, and measure it against human labels.
- Native-speaker review of explanations in Swiss national languages and in the most common migrant languages, and read-aloud audio for low-literacy users.
- PDF upload and an e-consent integration (for example a REDCap module), so a study team can offer ClearConsent next to the original form.

License

Code under Apache License 2.0. Documentation and data under Creative Commons Attribution 4.0 (CC-BY-4.0). All Hack Apertus projects are open-sourced.

References

- Swiss AI Initiative. Apertus v1.5 model cards, https://huggingface.co/swiss-ai/Apertus-v1.5-8B and https://huggingface.co/swiss-ai/Apertus-v1.5-70B
- Public AI Inference Utility. Developer documentation and model catalog, https://platform.publicai.co/models
- vLLM project. OpenAI-compatible server, https://docs.vllm.ai